Honours Project Report

Louis-Marie Matthews

April 2018

Abstract

U2F is at the present time the most recent two-factor authentication mechanism. In this paper I create a system to compare the usability of different security strategies in a peer-reviewable manner and use it to evaluate the usability of it compared to traditional complex password strategies. In addition, I develop a PHP library to easily add authentication to a PHP project. The user study reveals U2F with a simple password is 20% faster than a complex password and the NASA-TLX workload is 29% better. In addition, 87.5% of 25 surveyed PHP developers think that the library has its use.

1 Introduction

Pearman *et al.* describe passwords as a central component of online authentication [17]. However, they suffer from very important issues. Password authentication is classified among the What-You-Know (WYK) authentication mechanisms, which means it only requires information (i.e. the password). As a result, anyone with the knowledge of that information is able to impersonate the legitimate owner. The security of password authentication is made even worse by two other facts. Pearman *et al.* found that users often do not choose secure passwords, and they also overwhelmingly reuse their passwords, partly or entirely, across different websites [17]. This both gives an attacker more chance to get access to a password, and it makes the consequences of a stolen password worse, as accessing a stolen password on one online service would give the attacker the possibility to authenticate on other online services. Weak passwords also increase the feasibility of offline attacks, where an attacker attempts to extract passwords from leaked cryptographic hashes.

Guidelines for strong passwords do exist to mitigate against the aforementioned issues and are usually enforced online, especially on security-critical online services such as banking websites [17]. The guidelines for strong passwords usually consist of a minimum length such as 8 characters, and the presence of different character types. In addition, passwords must be unique for each account. Some security policies also set an expiry date for passwords, obliging their users to create a new one after a certain amount of time.

These rules however do lead to a poorer usability, and put an unrealistic workload on the user's memory [18] (unless a password manager is used). Most users do not choose strong passwords, and overwhelmingly reuse them across different websites [17]. The additional security that password expiry policies are questionable, and the new passwords are usually predictable. Strict security policies with a password expiry date and password complexity requirements lead to users attempt to circumvent it [12] (e.g. by writing down the password on a post-it). These guidelines are actually discouraged despite being widely enforced.

Besides, even if these guidelines were strictly followed by users, it still does not protect against attackers directly stealing passwords from users (e.g. by phishing). Users also tend to forget their passwords. Websites usually provide password recovery mechanisms, such as security questions. However, security questions can lower the overall security of the website. Rabkin observed that security questions used on banking websites often offer weak security [18].

Two-factor authentication (abbreviated as 2FA) is an attempt to augment the security of online authentication by asking the user to complete an additional step, for example when logging in from a new device for the first time or when performing a secure operation such as a bank transfer. In some cases, it takes the form of a secret code sent to the user's email address. However, overwhelmingly, it consists in generating and sending a one-time password (OTP) to the user via an alternative channel (the SMS channel, as opposed to the Internet challenge), and requiring the user to enter that code to authenticate.

1.1 Security Model

2FA provides a fallback in case the credentials of the user get compromised. If an attacker manages to get access to the user's credentials, the 2FA mechanism, if any, will prevent the attacker from getting access to the user's account. I will discuss in the following sections the additional security 2FA mechanisms can bring (e.g. against phishing).

2FA can also be used to protect against malicious host devices or mitigate against the risks of using one. (A malicious host device can be an infected computer.) This is not considered in this paper. Additionally, in the security model of this paper, only server breaches that reveal information to attackers (whether with or without the server knowing) are considered.

1.2 Current 2FA mechanisms

In this section I describe the most commonly found 2FA mechanisms. Most are OTP-Based 2FA mechanisms. OTP-Based 2FA mechanisms consist in requiring the user to enter a one-time password (OTP), sometimes communicated via a secure channel, to the server when attempting to authenticate.

1.2.1 SMS-based 2FA SMS-based 2FA relies on the mobile network to securely send the user a randomly-generated OTP via an alternative channel (the SMS network) to a device the

user owns (their mobile phone) [15]. The OTP is generated randomly on the server side and should be unpredictable for a potential attacker.

1.2.2 Application-Based 2FA Some online services base their 2FA mechanism on a mobile application. This mechanism relies on time-based OTPs (TOTP). An application on the user's mobile phone shares an cryptogtraphic key with the online service. This shared key is obtained when the user sets up Application-Based 2FA for their account on the online service. It usually involves the user flashing a QR code which transmits the secret key generated by the online service to the 2FA application. Afterwards, the encryption key is no longer exchanged [15]. To log in or to perform secure operations, the user can be asked to enter the current TOTP generated by their application. This requires the online service and the user's mobile phone to have their clock roughly synchronised, because the TOTP is generated from the encryption key and from the current time (with a certain precision, e.g. 10 seconds). This makes it impossible for an attacker not knowing the secret key to generate a valid OTP.

1.2.3 OTP Security Devices There exists devices specifically and exclusively meant to be used for 2FA. These will be called Security Devices in the rest of the paper. OTP Security Devices are security devices that are based on OTPs. They can take the form of a card reader that generates an OTP when the user's debit card is inserted and its PIN entered, or a Nitrokey. A Nitrokey can perform OTP-Based 2FA for multiple websites, using either HOTP (OTP based on an encryption key and a counter which should stay synchronised with the server) or TOTP. The security device just needs to be set up with the HTOP/TOTP encryption key of the online service, and can be used afterwards to generate valid OTPs.

1.2.4 Push-notification The **Push-Notification 2FA** mechanism does not rely on OTPs. Instead, the user is simply asked to confirm their authentication request by pressing a button on their mobile phone. This requires the mobile phone to be authenticated to the server. This is usually a feature built-in to the mobile phone OS. In practice, this means the user needs to associate an account they have on a certain online service with their mobile phone by logging in with this account on their mobile phone first (e.g. an Apple account on an Iphone or a Google account on an Android phone). However, this 2FA mechanism is only implemented by Apple and Google (on the Iphones and Android devices, respectively) as it is a OS built-in feature. For this reason, I will not consider it any further.

1.3 Potential issues with current 2FA mechanisms

All these 2FA mechanisms present potential issues that in some cases makes them insecure.

1.3.1 The SMS network This only applies to SMS-Based 2FA. SMS-Based 2FA relies on a separate channel, the SMS network. The use of a separate channel should increase the security of this 2FA mechanism, assuming the channel is secure. However this is not the case for SMS-Based 2FA. SMS can be read by government agencies, by operators or by attackers breaching the operator servers.

1.3.2 Effort This only applies to OTP-Based 2FA mechanisms. They require the user to copy and paste a random string of digits (or letters, as in the case of Yahoo!) every time the online service requires them to authenticate. They often have to do it manually (entering key by key, or copy pasting in some cases, or automated in some cases) which is a lengthy, error-prone and mentally-demanding process.

1.3.3 Latency and Network Issues In the case of SMS-Based 2FA, users have to wait for their mobile phone to receive the SMS containing the OTP (which can take several minutes, or even fail to reach the mobile phone in case of network failure).

1.3.4 Single point of failure This only applies to Application-Based 2FA and OTP Security Keys. HOTPs and TOTPs (the two types of OTPs) are generated using a secret key. This secret key is shared between the 2FA device and the online service. This means that, in the event of a server breach, an attacker could be able to access 2FA secret keys and as a result bypass 2FA for any user of the online service if this the breach is not acted upon in a timely manner.

1.3.5 Phishing This only applies to OTP-Based 2FA mechanisms. OTPs, as their name imply, are essentially standard passwords, the only difference being that they are generated automatically and used only once. As for standard passwords, this makes the OTP-Based 2FA mechanism vulnerable to phishing attacks.

1.3.6 Compromised Mobile Phone This only applies to SMS-Based 2FA. In an effort to offer a better and more consistent user experience between the computer and the mobile phone, Apple and Google introduced synchronisation features. Apple Continuum allows a user to synchronise their SMSs across their devices (e.g. laptop and mobile phone). Previous Iphones and current Android devices allow the remote installation of mobile applications from the user's Apple or Google account. These synchronisation features weakened the security brought by using a separate device for online authentication, and made SMS-Based 2FA mechanisms insecure [9]. As a result of these synchronisation featyres, an attacker who breached into the user's Apple account is able to read SMS from the user's mobile phone, and bypass SMS-Based 2FA verification steps for the user. Similarly, an attacker who breached into the user's Android account (or Apple account if the iOs version used is old) is able to install a mobile application on the user's mobile phone, and this mobile application is then able to almost automatically read all SMSs as well.

1.3.7 Cost This only applies to to SMS-Based 2FA. This 2FA mechanism requires the online service to be able to send a

potentially large numbers of SMSs at any time, which can have a significant cost.

1.3.8 Anonymity In addition, SMS-Based 2FA require the user to give up personal, identifying information to the online service (i.e.. their phone number). This is a potential issue for the user's privacy, in case of a server breach or a malicious website. This makes a server breach even more dangerous.

1.3.9 Lack of Inter-Operability Even for the same type of 2FA mechanisms, implementations can vary and be incompatible. For example, SMS-Based 2FA does not define a standard format for the content of SMS transmitting OTPs. There is also not standard way of communicating HOTPs or TOTPs from the client to the server. This prevents the interoperability of certain features, such as the computer or the mobile phone filling the field or the clipboard automatically from the SMS or from the OTP security device.

1.3.10 Separate device Finally, purely from a usability perspective, all the presented 2FA mechanisms rely on a separate 2FA device that can run out of battery (except in the case of most Security Devices that can be plugged in to the computer's USB port).

1.4 Contribution of this paper

In this paper, I examine another 2FA mechanism called U2F. U2F is very recent compared to most mechanisms presented so far, and functions very differently. Section 2 gives an overview at the U2F 2FA mechanism, including its protocols and its security properties. Section 3 explains what are the goals of this project and how they were determined. Section 4 is a thorough description of the two software products developed in an attempt to achieve the goals set earlier. Similarly, Section 5 describes the user study that was executed to address part of the goals set in Section 3, as well as its analysed results and conclusions. Section 6 explains the approach that was used to develop the software products. Section 7 critically assesses what was achieved, to what extent were goals met, and suggest areas of further research and improvement. Finally, Section 8 summarises the essential points of the project.

2 The U2F-Based 2FA Mechanism

I give a brief overview of the FIDO U2F 1.2[1] specification in this section.

The FIDO U2F specification is an industry standard developed and maintained by the FIDO Alliance, a consortium of online service and security companies including Google, Yubico, Microsoft, and StrongAuth.inc. It creates standards for open standards to improve the user experience of strong online authentication one and make seamless across online services for users. One such standard is the FIFO U2F specification.

Figure 1: A Yubico blue U2F Device attached to a key ring

The FIDO U2F specification defines two processes: the **U2F Registration** process and the **U2F Authentication** process. In both cases, it requires HTTPS.

I use the following terms to refer to the actors part of both processes.

Definition 2.1. One central component in both U2F processes is a piece of hardware called a **U2F Device**[2]. U2F Devices are small devices that attach to a key ring (Figure 1). They do not rely on a battery, and are usually designed to be tamper-resistant [15]. They can be plugged in using USB, but some U2F Devices also support NFC and Bluetooth.

A U2F Device can be thought of as a physical password. Let's suppose a fictitious website supports U2F. When the user creates their account, they choose their username and password. On the next page they are asked to register a U2F Device. (This is the U2F Registration Process.) When the user attempts to log in from a new computer to perform a secure operation, they are asked to activate the U2F Device they registered (the U2F Authentication).

Definition 2.2. The **Host Device** (referred to as the FIDO client in the FIDO specifications) is the device on which the user is attempting to register a U2F Device, or to authenticate using one of their U2F Devices. It can be a laptop, a desktop or a mobile phone.

Definition 2.3. The **Online Service** (referred to as the Relying Party in the FIDO specifications) is the website to which the user is registering their U2F Device and authenticating using it.

Definition 2.4. The **User** is the human attempting to register a U2F Device, or to authenticate using one.

In addition, any Online Service possesses an Application ID.

Definition 2.5. In the case of a website, The **Application ID** of an Online Service is its web origin (protocol, which needs to be HTTPS, domain and port) [14]. Mobile applications have a similar identifier. If the Online Service is made up of several "facets" (e.g. a mobile application, another domain), the Application ID can be the origin of a page that lists all these facets.

[1] At the time of writing, the FIDO 2 project is ongoing, and the WebAuthn specification which ultimately should supersede the FIDO U2F specification is in a Work-In-Progress status. This effectively means some technical details described in this section will relatively soon turn obsolete. However, WebAuthn will be backward-compatible with U2F Registrations generated with the old specification, and the general principles of U2F-Based Authentication will still hold.

[2] The specification actually uses the term "U2F Token", which also includes software implementations in addition to physical devices. This practice however, is discouraged [19], and I only consider U2F Devices in this paper.

2.1 U2F Registration

I explain the U2F Registration process in this section with an example. Many irrelevant technical implementation details are omitted such as the precise format and content of the messages or the hashing algorithms used. These are however thoroughly described by the FIDO Alliance [6].

Let's assume a bank added support for U2F on its online banking website, and that a User wants to register a U2F Device on their account to improve its security. The User accesses the online banking website from their laptop.

In this case, the Online Service is the online banking website, and the Host Device is the User's laptop.

The User would first need to register their U2F Device. In order to do so, the Online Service would provide a page for that purpose (e.g. https://bank.com/add-u2f-device). When the User accesses that page, the Online Service generates a U2F Register Request. A U2F Register Request is made of a Version (e.g. "U2F_V2" for U2F v1.2) and a Challenge, which is a random string. The Online Service stores the generated U2F Register Request somehow (e.g. in session) as it will need to retrieve it later.

The Online Service returns an HTTP response containing JavaScript code in which there is a call to `u2f.register()`[3]. To this function is passed the Application ID of the Online Service (Definition 2.5), the previously-generated Register Request[4], a callback to be executed when the response from the U2F Device is received or when the timeout has been exceeded, and optionally, a timeout. Additionally, if the User already has U2F Devices registered with the Online Service, they should be passed to `u2f.register()` as well to prevent the User from registering the same U2F Device twice.

All of this of course is invisible to the User. The page displays instructions to the User. (E.g. "Plug your U2F device in.") This is what the User does. When his or her U2F device is plugged in, it starts blinking. This is because the Host Device executed the `u2f.register()` function described above and passed to the U2F Device a hash of the Application ID and a hash of the Client Data [5].

The Client Data contains the type of the Request (whether the Request is a Register or a Sign Request), the Challenge that was passed to `u2f.register()`, and the Application ID of the Online Service [6].

The User presses the button on the U2F Device. The U2F Device perceives this as a gesture of approval. It generates and stores a new public and private key pair associated with the Application ID of the Online Service and with a Key Handle (generated in the process as well). It then returns a U2F Register Response to the Host Device. The U2F Register Response contains the Public Key that was generated, the Key Handle, and a signature of the message (complete with the hash of the Client Data) using the U2F Device attestation certificate[7].

The Host Device calls the callback defined in the call to `u2f.register()` with the U2F Register Response and the Client Data. The callback fills a hidden HTML form with the serialisation of the U2F Register Response and triggers the form submission.

The Online Service receives the submission, and validates the U2F Register Response. (Is the format valid? Is it an error?) It checks the Client Data. Both the Online Service and the Host Device have sufficient information to compute the Client Data. If the Online Service finds a Client Data that does not match the one sent back by the Host Device, it indicates a MitB (Man-in-the-Browser) attack: the Host Device was communicating with the malicious website and generated the Client Data based on the Application ID of that website. If the malicious website attempts to do a MitB attack by receiving the Client Data from the Host Device and passing it to the Online Service, the Online Service would notice the Client Data does not match. In other words, the Online Service is able to notice the Host Device is not talking directly to the Online Service.

The Online Service also checks the U2F Register Response against the U2F Register Request that it generated before (and possibly stored in session). If the U2F Register Response the Online Service received is valid (not a timeout, correct Client Data, Client Data correctly signed, etc.), it stores a new U2F Registration (also called, a U2F Registered Key) somewhere (e.g. in a database).

2.2 U2F Authentication

We are still in the example from Sub-Section 2.1. From the user's point of view, the process is remarkably similar. The User requests a page, e.g. "https://bank.com/login". The server generates one Challenge (a random string)[8], and stores it somehow (e.g. in session). It also find the Key Handles of the U2F Registrations of the User. A U2F Registration corresponds to a U2F Device the User registered with the Online Service[9].

The Online Service returns a page to the User. It contains JavaScript code. In particular, a call to the function `u2f.sign()` is made. To this function is passed the Application ID of the Online Service, the Challenge, the found Key Handles, a callback and optionally a timeout.

As for U2F Registration, the Host Device passes a hash of the Client Data. It contains the type of the Request (in this case it is a Sign Request), the Challenge, and the Application ID. It also gives it the hash of the Application ID. However, unlike in the Registration process, it also gives it a Key Handle.

The U2F Device determines whether it has the specified Key Handle and, if it does, whether the Key Handle is associated with the given Application ID. If both conditions are not true, it returns the same error. It does not indicate whether it

[3] Note that the use of JavaScript is not necessary when dealing with U2F, and a low-level MessagePort API can be used instead. This is however not recommended [5]. The JavaScript high-level API is both easier to use and is less likely to change over time.

[4] The specification allows for multiple U2F Register Requests to be passed. To keep the explanation simple, I will not consider this edge case. When multiple U2F Register Requests are present, the Host Device will choose the first one with a supported version, and ignore the rest.

[5] It is also called the "challenge parameter", but this is ambiguous with the Challenge that is passed when calling `u2f.register()`.

[6] The public key of the ChannelID is also included if the TLS connection is using ChannelID.

[7] There are other parameters omitted for simplicity: the attestation certificate itself, reserved bytes, legacy bytes, etc.

[8] This is an optimisation from the FIDO U2F 1.0 specification which required one challenge per U2F Registration

[9] The Online Service can never guarantee that each U2F Registration were generated by a different U2F Device, it can only dissuade Users to do so. This is on purpose: U2F Devices are designed to be not identifiable by Online Services.

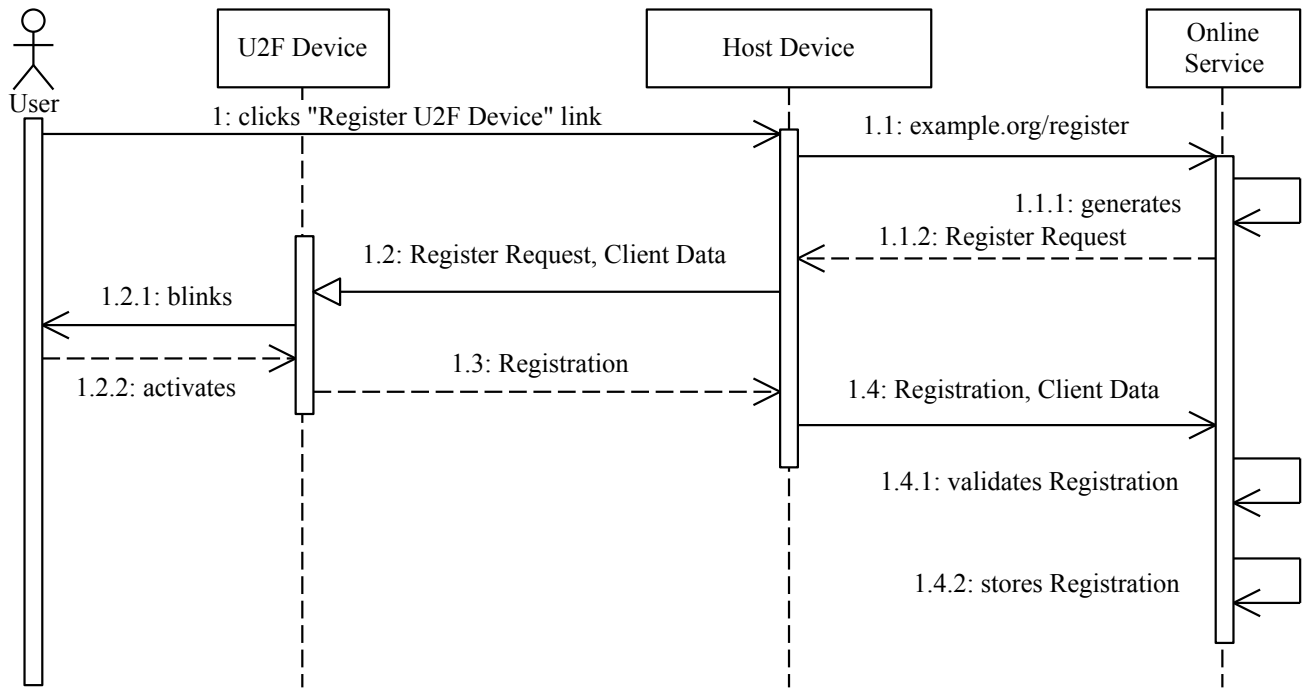

Figure 2: Overview of the U2F Registration Process

did not recognise the Key Handle, or whether the Key Handle is not associated with the Application ID that was given. This prevents Online Services to learn whether a certain User holds an account at another Online Service using leaked (or otherwise) Key Handles from that Online Service.

Otherwise, it returns a Sign Response, which contains the Challenge signed with the Registration's private key, and a counter.

In both cases, the U2F Device waited for the User's approval before sending its response back to the Host Device.

The Sign Response is sent to the Online Service along with a hash of the Client Data (for the same reason as in the Registration process, see Section 2.2).

The Online Service verifies the Sign Response has the valid format and is not an error message. It also checks that the Client Data is correct, and that that the Challenge was properly signed (using the public key it holds for the Key Handle).

Finally, it also checks for the counter. The counter is the number of times the U2F Registration was used. If the counter the Online Service holds for the User Registration is less than the own part of the Sign Response, this is normal, and the Online Service updates the value of the counter it has (e.g. in the database). Otherwise, if the counter in the Sign Response is smaller than the one the Online Service holds for the U2F Registration, it indicated a cloning attack. The U2F device of the user was cloned, but it was not used as often as the real U2F device. As a result, its counter is lower. This allows the Online Service to detect cloned U2F devices and take appropriate measures (e.g. lock the User account).

2.3 Advantages of U2F

U2F addresses all the issues mentioned in sub-section 1.3.

- It does not rely on an insecure channel (Issue 1.3.1).
- It requires minimal effort, which is just pressing a button on a U2F Device (Issue 1.3.2).
- There is no additional latency (Issue 1.3.3).
- If the server is breached, attackers would not be able to use the stolen information to complete the U2F authentication (Issue 1.3.4). This is because U2F Registrations are scoped to a particular Application ID.
- U2F makes phishing attacks impossible (Issue 1.3.5). The Host Device sends back the Client Data, which includes the Application ID of the Online Service the Host Device is talking to. If this Application ID differs from the one of the legitimate Online Service, the legitimate Online Service will refuse to perform the operation (e.g. logging in, transferring money, etc).
- It does not rely on a potentially compromised mobile phone (Issue 1.3.6).
- U2F does not incur any additional costs for the online service, nor any infrastructure (Issue 1.3.7). U2F is simple to implement and cheap for the User [15]. Only the User is required to have a U2F Device, but these are cheap and reusable across online services, and the cost is negligible.
- U2F protects anonymity and privacy (1.3.8). U2F Devices are not identifiable remotely by the responses they send, either Register Responses or Sign Responses.
- It is an open standard designed with interoperability in mind (Issue 1.3.9).
- U2F Devices are small in size and do not have a battery (Issue 1.3.10).

Baghdasaryan *et al.* list many other security properties of U2F [2]. Listing them here would offer little interest.

2.4 Related Work

The FIDO alliance provides another overview of the specification from a different angle [19]. The JavaScript API [5] and the description of the format of the messages exchanged between the Online Service and the U2F Device [6] are both also very helpful in understanding the finer details of the general specification. They are also an essential read to properly add support for U2F to a website. An extensive review of the threat model, security goals and mechanisms and potential attacks against the U2F protocol is also available [2]. The FIDO alliance also offers advice on properly implementing the logic of a U2F Device [4], which contains interesting details even for readers not interested in creating their own U2F Device implementation. More information on the concept of Application ID and Application Facet, two concepts originating from the FIDO specifications, can also be found [14]. A glossary is also made available for reference [3]. Lang *et al* [15] provide a general yet relatively detailed and readable description of the FIDO U2F 1.0[10] specification and compare it with other second-factor authentication mechanisms both in terms of usability and security.

3 Specification

3.1 Evolution of the Goals

At first, the project was aimed at Security Devices in general. This included Nitrokeys[11] with a built-in password management system, HOTP/TOTP generator, and GnuPG email encryption. After research done on the topic, I decided to limit it to U2F. This was because, unlike general-purpose Security Devices such as Nitrokeys, the problem space with U2F is more precisely defined. U2F is a standardised and novel technology. Nitrokeys are merely implementations of existing technologies on secure, external hardware, and as such does not bring much novelty. I also thought U2F had the potential to transform online authentication. Several major websites now have support for U2F including Dropbox, Facebook and Google, and the W3C is currently working on a new standard based upon the U2F technology. Despite its potential however, it still remains a rare sight in shops and websites rarely have support for U2F. This I believe adds to the interest of directing my project towards U2F, as it is an yet-to-emerge technology with a lot of potential.

After deciding to orient my project in the U2F direction, I determined a precise goal for the project: Creating a application supporting U2F that would allow web developers to create websites with built-in support for U2F. The software to develop would take the form of a script. The script would be associated with a configuration file, which would determine options for the website to generate, e.g. whether it should create an administration interface, etc. This first goal, being well-defined, allowed me to move from the general research phase to the development phase. I soon was able to get a working website supporting U2F.

However, I realised at the same time that the goal of the project had several issues. The project essentially consisted in creating a CMS with support for U2F. Creating a complete CMS is a hard task for only one developer, especially if the developer cannot work on it full-time, which was my case. Plenty of existing CMSs already exist such as Drupal, PrestaShop and WordPress. These CMSs are very popular and are of good quality. Some of them already support U2F[12]. Besides, the task of creating a CMS was so tedious compared to the task of making it support U2F that it effectively shifted the project's problem space to CMS development instead of U2F.

I needed to change the direction the project was taking. Instead, I decided to limit myself to the creation of a website with simple, commonly found features such as registering and logging, all of this with support for U2F. The website would serve as a template allowing other developers to use that template to start building new websites with support for U2F out-of-the-box. It did not take me long to realise that this approach also had severe issues: most developers use web frameworks when creating websites. For PHP, the most popular ones are Symfony, Laravel and CakePHP. As a result, I could not expect developers to use the software product I was developing. In addition, my software project couldn't be used for existing websites, as it only served as a starting point.

For this reason, I limited the project even more to the creation of a website with support for U2F as well as features traditionally found on most websites. The website this time would serve as an educational example of the use of U2F. In addition, a user study would be made on the website to evaluate its usability and make necessary adjustments. This remained the goal of my project for several months.

I however became aware that it was possible to extract the logic related to authentication and registration (including U2F authentication and registration) into a separate library. There already existed PHP libraries that provide classes and functions to implement the U2F protocol such as Firehed U2F[13]. However, any application using these libraries still needs to deal with a certain number of details specifically related to U2F such as displaying pages embedding the correct JavaScript code that would trigger a U2F Register Request for instance.

At first, I attempted to extract the U2F authentication into a library that would function as an additional HTTP service. The application and the library would communicate through the user's browser with HTTP requests. They would run on the same or on separate server(s) and use pre-deployed cryptographic keys to ensure the security of their communication. This however would require a tedious set-up for any developer willing to use this library for their own web applications. Besides, a secure protocol was necessary to allow the application and the library to communicate using the user as a relay (and without the user being able to tamper with the data or perform replay attacks). Designing such a protocol is a very hard task, with existing protocols such as OAuth2 difficult to implement properly.

Instead, I decided to create a library that would function as

[10] The version 1.0 and 1.2 of the FIDO U2F specification only slightly differs in regards to the general process.

[11] There exists a Nitrokey FIDO U2F device, but this is not considered here.

[12] `https://github.com/georgestephanis/two-factor`

[13] `https://github.com/Firehed/u2f-php`

an HTTP middleware. The application would pass it the incoming HTTP request, and the library would return an HTTP response. This would completely abstract the complexity U2F from the application. Along with this library, I decided to create an example application to prove that the library is functional. I also planned to use the example application to run a user study and compare different U2F security strategies: one with 2 U2F Devices versus one with three U2F Devices. This was designed to help developers make informed decisions when designing a U2F Security Strategy for their web application. (E.g. A U2F Security Strategy would answer questions similar to: How many keys to ask the user when registering? How many U2F devices to perform a secure operation? Does it require a password as well?) However, comparing the usability of a two-U2F-Device versus one with three made little sense as users would certainly find it easier to use a website that requires less U2F devices. In the end, I decided that it would provide more interest to compare a traditional authentication mechanism, such as complex passwords, vs U2F.

3.2 Final Goals

In January, 2018, my goals were in their definitive form. They are stated below.

Goal 3.1. Create a library that abstract U2F authentication. The library must function as a middleware that is easily inserted into new or existing web applications, and allow users to authenticate using their U2F device.

Goal 3.2. Create an example web application that uses this library to allow its users to use U2F. The web application must also support U2F registration, passwords, and other features, mock generally found on most websites (logging in, registering, a dummy money transfer page).

Goal 3.3. Compare the usability of complex passwords versus using U2F.

3.3 Sprints and Gantt chart

Development started in November, 2017, and a Scrum methodology was used. In Sprint 0, U2F was first implemented in PHP. This took time as I needed to familiarise myself with the U2F technology and its PHP implementation. In Sprint 1, it was then integrated with Symfony and the development workflow was set up (which included setting up Docker). Authentication and registration features were then added in Sprint 2. Afterwards, in Sprint 3 and 4, an administration features were added. In Sprint 5, the middleware library was created , while the web application was improved. Appendix A.1 is the Gantt chart that was used.

4 Description of the final product

Two software products were developed, *auth-abstractor* and *security-comparator*.

4.1 *auth-abstractor*

auth-abstractor is a library that works with any PHP application (with PHP > 7.1). It can easily be installed using Composer[14], the most commonly used package manager for PHP dependencies. *auth-abstractor* provides an almost complete abstraction of authentication and registration for supported mechanisms. These mechanisms include: password authentication and registration, U2F authentication and registration, and email two-factor authentication (where a OTP is sent to the user's email address).

An application that uses *auth-abstractor* does not need to create specific views (i.e. templates for displaying an HTML page) for the login or registration page. Neither does it need to take care, for instance, of hashing and salting the passwords, or to generate U2F sign or register requests. In addition, *auth-abstractor* is application-agnostic. It can work with any web-framework or even without any web framework at all. *auth-abstractor* is simply not aware of the implementation details of the application. Finally, its set of supported mechanisms can easily be extended.

Figure 3 gives an overview of the use of *auth-abstractor* by a fictitious web application. Note that in the example given, the web application's `LoginHttpController` stores the `AuthenticationProcess` object in session. This is something that *auth-abstractor* is not aware of. This separation of concerns ensures *auth-abstractor* is not tied to a specific storage system, in this case the PHP native session storage system.

Two terms are central to *auth-abstractor*: Authentication Processes and Challenges.

Definition 4.1. An **Authentication Process** is a series of steps that the user is asked to perform by the web application in order for a certain event to happen. The event can be: "The user becomes logged in.", "The user account becomes created", or even "The user account gets deleted". It is not limited to authentication in its traditional form, and can include a series a forms that when submitted correctly allows a visitor to create an account on the website, or allows a logged-in member of the website to change their password. In this case, talking of a Registration Process or Account Modification Process is more accurate. However, for simplicity purposes, I use the term Authentication Process to either refer to both of these types of processes: processes that let the user perform an action (such as creating an account or changing their password), and processes that attempt to authenticate a user or verify their identity. Note that processes of both types can be mixed together in one single process.

Definition 4.2. A **Challenge** is step in an Authentication Process. It can be a form that asks the user for their username and password to verify the user's identity, it can also be a similar form to instead let the user create an account. It can also be a page on which the user is asked to activate their U2F device registered with the website to proceed.

4.1.1 Instantiating the authentication kernel The `AuthenticationKernel` object needs to be initialised with an instance of `IApplicationConfiguration`. This

[14] `https://getcomposer.org`

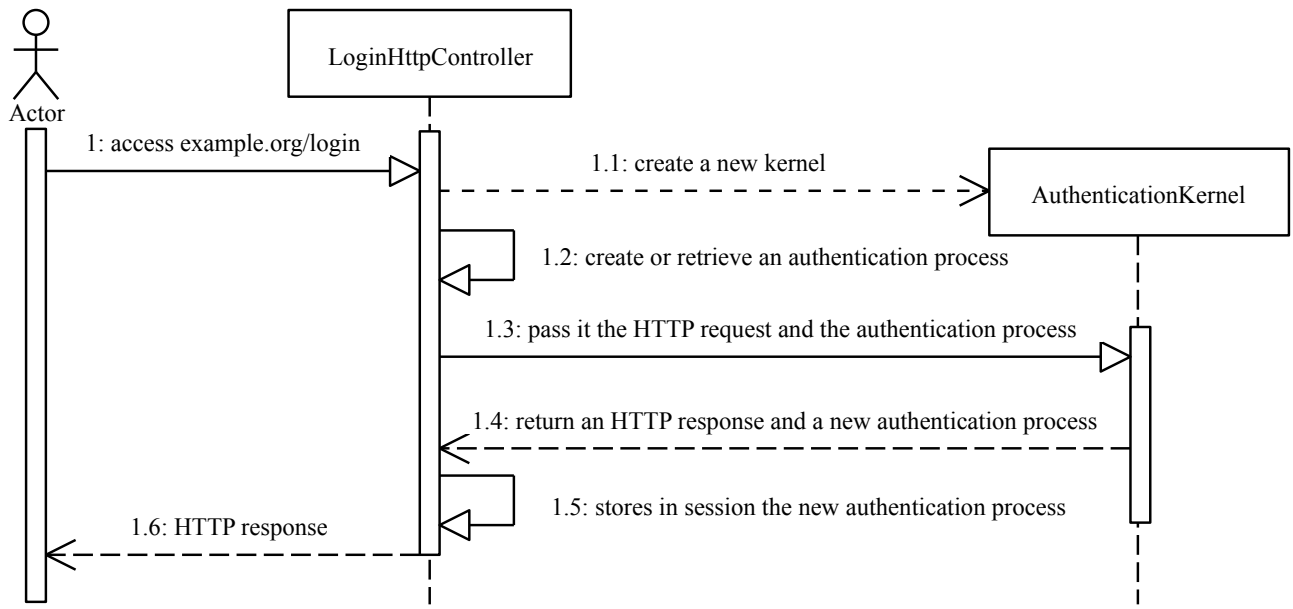

Figure 3: Simple overview of a use of *auth-abstractor*

interface defines methods that are then called by other objects part of *auth-abstractor*. For instance, the U2F authentication challenge calls `getAppId()` on the application configuration object, which must return the U2F Application ID of the web application. `getMember()` is used by most challenges in *auth-abstractor*, including the credential authentication challenge. (The challenge that verifies the user's identity by asking them for their username and password.) The challenge can call this method and pass it the username that the user provided in the form. It then checks the password submitted by the user against the the hash returned by `getHashedPassword()`.

Implementing `IApplicationConfiguration` is the responsibility of the application. However, implementing an interface can be a relatively cumbersome process, especially if the developer simply wants to try out *auth-abstractor*. For this reason, *auth-abstractor* provides a convenience implementation, `ApplicationConfiguration`, making it possible to instantiate `AuthenticationKernel` in the following manner:

```php
<?php
$config = new ApplicationConfiguration(
    "https://example.org",
    "https://example.org/assets",
    function (string $username): ?IMember {
        // Finds member from DB, or null
        return $member;
    }
));
$kernel = new AuthenticationKernel($config);
```

`ApplicationConfiguration` does not offer all the options that implementing `IApplicationConfiguration` provides however, but this is by design. `ApplicationConfiguration` is made voluntarily more simple but less powerful.

4.2 Instantiating the authentication process

Authentication Processes need to be able to remember data between HTTP requests. For instance, they needs to remember what the current Challenge is[15]. This is the role of the `AuthenticationProcess` object. It is an object which only responsibility is to contain data that is reused and updated across HTTP requests and Challenges of the same Authentication Process.

For example, consider an Authentication Process that allows a user to create an account. The user is asked on the first page to enter a username and a password. On the second page, the user is asked to register a U2F device. If the user's U2F response is valid, an account is created. At this point the web application needs to remember what as the username and the password that the user specified in the first Challenge. In *auth-abstractor*, this would all be stored in the `AuthenticationKernel` object. There is only one such object per Authentication Process.

auth-abstractor provides a factory for creating `AuthenticationProcess` for more convenience. The factory is called `AuthenticationProcessFactory`. It can be accessed from the kernel by calling `$kernel->getAuthenticationProcessFactory()`.

The Authentication Process Factory defines a `createProcess()` method. Several arguments can be passed to `createProcess()`, including whether the user is already logged in or not (and if so, what their username is), the maximum number of failed attempts allowed[16], and any other additional and arbitrary data that will remain stored in the `AuthenticationProcess` object for the whole duration of the Authentication Process and after, so that Callbacks, described below, are alter be able to access this additional

[15] This cannot be guessed from the URL alone as an Authentication Process in *auth-abstractor* uses the same URL for all Challenges.

[16] This maximum number of attempts is not enforced for Challenges that do not attempt to prove the user's identity, e.g. the credential registration Challenge.

data.

The only required argument to call the `createProcess()` function is an array of Challenges (or rather, Challenge FQCNs[17]. Challenges are implementations of `IChallenge`. It is by the way completely possible for a web application to define its own challenges and pass them to the `createProcess()` method.

More technically speaking, `IChallenge` implementations are responsible for rendering and processing a specific step of the Authentication Process (or Registration, etc.). For example, `U2fChallenge` renders a page asking the user to plug their U2F Device in, and also determines if the U2F response it received was valid after the browser sends the response back. If this is the case, if the U2F response is valid, *auth-abstractor* adds a `PersistOperation` object to the `AuthenticationProcess` object. The `PersistOperation` object containing the U2F registration with the updated counter along with whether the the authentication succeeded or failed.

It is worth noting one more time *auth-abstractor* is not tied to any specific challenge. While it defines its own, it would accept any class that implements `IChallenge`. This makes it easily extendable.

4.2.1 Passing the request to the kernel

The method that `LoginHttpController` calls in Step 1.3 of Figure 3 is:

```php
<?php
public function processHttpRequest(
    ServerRequestInterface $httpRequest,
    AuthenticationProcess $process,
    IAuthenticationCallback $callback
): AuthentifierRespone
```

`$httpRequest` represents the current HTTP request and `$process` is the `AuthenticationProcess` object that was either created or retrieved from session by the web application, e.g. by the `LoginHttpController` in Figure 3.

`$callback` is an implementation of the `IAuthenticationCallback` interface that is part of *auth-abstractor*. I discuss Callbacks in the next section.

4.2.2 Callbacks

Callbacks are essential to any Authentication Process in *auth-abstractor*. Whenever an HTTP controller calls `processHttpRequest()`, it passes a callback to it. This callback is an implementation of `IAuthenticationCallback` which defines two methods, `handleSuccessfulProcess()` and `handleFailedProcess()`. They are called when the Authentication Process either fails or succeeds.

As is the case for `IApplicationConfiguration`, a convenience implementation of `IAuthenticationCallback` is provided. Creating one is very simple:

```php
<?php
$callback = new Callback(
    function ($authProcess) {
        return new Response("Too many attempts");
    },
    function ($authProcess) {
        return new Response("You're logged in");
    }
}
```

It is important at that point to note that *auth-abstractor* never directly modifies the state (i.e. the database, or the session of the web application) of the web application. Instead, *auth-abstractor* instantiates `PersistOperation` objects and stores them in the `AuthenticationProcess` object. These `PersistOperation` objects can be processed atomically at the end of a (successful) Authentication Process by the Callbacks.

`PersistOperation` have a type. If the `PersistOperation`'s type is `"CREATE"` and its entity is a new member, the application-defined callback will save the member's username and hashed password in the database.

4.2.3 Customisation

auth-abstractor uses the Twig library to render HTML pages. Twig is a templating engine. It allows the rendering of HTML pages with features such as variables, loops, inheritance, etc. For instance, is is possible to specify a base Twig file that contains the HTML code common to all pages, and more specific templates for each specific page. It also protects against XSS attacks.

`IApplicationConfiguration` defines the `getCustomTwigDir()` method. Implementations can return a valid path to a directory containing custom Twig templates that would be used in lieu of *auth-abstractor*'s templates[18], or it can return `null` if the application does not want to override any templates. While *auth-abstractor* uses Twig, it doesn't force the rest of the web application to use Twig as well. An application using Laravel could use Blade (another templating engine) instead, and would not be aware that one of its dependencies is using Twig to generate certain HTML pages.

It is also possible to specify the password requirements by implementing `getPwdSettings()`. This method needs to return an array determining the minimum number of characters, whether passwords need to contain special characters, numbers, or uppercase letters.

A one-page example of the use of *auth-abstractor* can be seen in Appendix E. The manual can be found at:

4.3 *security-comparator*

security-comparator is a web application that is designed to aid Human-Computer Interaction researchers to perform and peer review user studies comparing the user performances and the user experience of different security strategies.

Definition 4.3. In this paper, a **Security Strategy** is a set of rules that determines how the user is allowed to authenticate. For example, the security strategy could allow the user to authenticate using one password and one U2F device, or one password and one mobile phone on which an OTP will be sent.

[17] A FQCN (for "Fully Qualified Class Name") comprises the name of the class as well as its namespace.

[18] They would be overridden on a template-per-template basis, making it possible to override only certain templates and keep the other default ones.

4.3.1 Installation The source code of *security-comparator* can be accessed publicly from its repository[19]. The web application requires Docker and Docker Compose. Once the repository is cloned, the command `./bootstrap` should be executed from the working directory. When the command finishes, it is possible to access the web application at `https://172.16.238.10`. The web browser displays a warning the first time the website gets accessed because the TLS certificate is not signed by a certificate authority. This is normal. The TLS certificate used is a self-signed TLS certificate. The web application should be added to the web browser's exceptions.

4.3.2 Administration Once it is possible to access the web application, the researcher (i.e. the intended user of *security-comparator*) can log in as an administrator. The default username is "louis" and the default password is "hello". Once logged in with an administrator account, it is possible to access the administration dashboard. A link to it is present in the footer of every page of *security-comparator*. From the administration dashboard, the different administration panels are listed:

- **Security Score** displays a score associated with the current Security Strategy, and make it possible to specify custom Security Strategies and calculate their Security Score.
- **U2F Settings** allows the researcher to set security settings relating to U2F.
- **Credential Settings** allows the researcher to set security settings pertaining to passwords.
- **User study settings** allows the researcher to activate or deactivate the user study mode (i.e. start or stop recording performance metrics) as well as set the current participant's name or ID.)
- **See metrics** allows the researcher to see the collected performance metrics for each participant, and to export them to a CSV file. The number of errors the participant made is also displayed.
- **Export settings** allows the researcher to download a JSON file which contains the current configuration of the web application and of the user study (U2F settings, credential settings, etc).
- **Import settings** allows the researcher to import an external JSON configuration file.

In particular, the **U2F Settings** panel contains several options.

- **Allow U2F login** allows (or prevents) participants to log in (or from logging in) using their U2F Device.
- **Members' minimum number of U2F Devices** is the number of U2F Devices users are asked to have registered on the web application to use the web application. If they do not have enough U2F devices, they will be asked to register as many new U2F Devices as imposed by the Security Strategy before being able to use the web application normally.
- **Number of U2F Devices asked during registration** is self-explanatory. Note that this can be lower that the minimum number of U2F Devices above, which would allow users to create an account but prevent them from logging in without registering a new (or more) U2F Device(s) first.
- **Allow members to manage U2F Devices** determines if users are allowed to add or delete existing U2F Devices registered with their account.

When the user study mode is set to "On" from the administration panel, the website collects the time the user takes per page and saves it in the database under the current participant ID (set in the administration panel as well). These page metrics can then be downloaded to CSV files for analysis. If comparing two or more Security Strategies in a within-subject experiment, it is advised to include the level in the participant ID. E.g., for the first part of the evaluation, the participant ID would be set to "P428 - Password", and for the second part it would be set to "P428 - U2F" for example. This allows the two datasets to be downloaded separately.

4.3.3 Front-End The rest of the website mimics a fictitious banking website. It is possible to register or to log in. Once logged in, it is possible to pretend transferring money (by clicking on "Transfer money" in the navigation bar). It is also possible to:

- manage the user's U2F Devices (add or delete),
- change the user's password, and
- delete the user's account.

4.3.4 Security Strategy Score This section describes the method used to calculate a Security Strategy Score SS_k for a Security Strategy k.

k can be written as $k = \{m_1, ..., m_n\}$. $m_1, ..., m_n$ are the authentication mechanisms supported by k. This means that with the Security Strategy k, a user is able to log in using either m_1, ..., or m_n. To each mechanism m_i is associated a Mechanism Security Score MS_i.

Building on the premise that information security is as strong as its weakest link [1], we can deduce that $SS_k = min(MS_1, ..., MS_n)$.

Each mechanism m is a set of tuples $\langle c, j \rangle$, with c being the challenge definition and j the number of challenges. We have $c \in C$, C being the set of all the challenge definitions. A challenge definition represents a type of authentication step, such as entering as password or validating a U2F device. A challenge definition $c \in C$ has several properties:

1. An access resistance, which denotes how hard it is for an attacker to get access to the authenticator (e.g. the password, or the U2F device, etc).

[19] urlhttps://github.com/matthewslouismarie/security-comparator

2. A phishing resistance, which denotes how bad the consequences of the user visiting and interacting with a phishing website can be for the security of the authenticator associated with the challenge and the user. If it is 0, this means that visiting the phishing website is enough to jeopardise the security of the challenge.

3. A reproducibility resistance denotes how hard it is for the attacker to replicate the authenticator if he or she has access to it.

4. A server leak resistance, which denotes how badly the security of the authenticator is affected in case of a server information leak. If the score is 0, it means the authenticator can no longer be used to distinguish between the attacker and the victim.

5. A duplication factor. The duplication factor determines how much security is added when adding to the mechanism another challenge of the same type.

All these properties take a value between 0 and 1, with 0 meaning not secure at all and 1 meaning completely secure. From these properties, we can calculate the Security Score of the Challenge CS_c as the sum of the value of the properties mentioned above multiplied by the number of factors. The number of factors can be calculated as the number of different challenge definitions present in the mechanism m, to which is added each remaining duplicate challenge definition's duplication factor. So, for a mechanism made requiring one password and two U2F devices, with the duplication factor of U2F devices set at 0.6, the number of factors N would be $N = 1 + 1 + 0.6 = 2.6$.

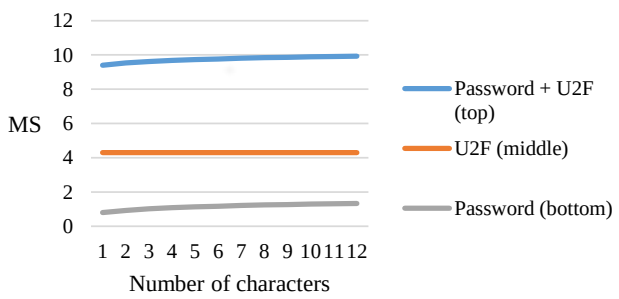

Figure 4: The Security Score of different Mechanisms

Figure 4 compares the Security Score of a Mechanism based using solely passwords against one based solely on U2F devices and on based on both a password and a U2F device. (This means users would be required to enter their password and to activate their U2F device, it would not be a choice.) U2F alone is much more secure than the use of long passwords alone, even if they are long. However, the use of U2F and passwords in the same Security Strategy offer the best security. These findings seem reasonable, reasonably confirming the developed model.

Appendix G references the location of the code implementation of the formula. *security-comparator* allows to increase or decrease the importance of each property. By default, they all have the same importance. The implementation of the password challenge definition gives an increasing resistance to guessability and to server leak as the minimum numbers of characters augments.

5 Evaluation

Using *security-comparator*, a user study was run in order to compare the usability and the efficiency of two security strategies: a security strategy based on complex passwords, and a security strategy based on simple passwords and U2F devices.

5.1 Methodology

5.1.1 Participants The target population are educated young adults. Participants were selected from the local university student population, mainly because it was convenient forme to recruit participants from this population. In total, 34 participants took part in the user study.

The results collected from the first 11 participants who took part in the user study unfortunately had to be discarded (see Section 5.4 for an explanation). Of the remaining 23 participants, the results of two of them were also discarded because the procedure was not strictly followed. (One used an external screen, the other filled their NASA-TLX form after the trial.) Finally, the results of one last participant were discarded because the participant's age was too different from the rest of the participants ($\sigma \approx 1.16$ without the participant, $\sigma \approx 13.16$ with the participant).

In the end, the results of 20 of the participants were kept. This met the minimum number of participants for an experiment involving NASA-TLX forms [16]. The mean age and associated standard derivation were $\bar{x} = 21.6$ and $\sigma \approx 1.16$. Participants covered a wide range of IT abilities (as seen in Figure 5). 13 of the participants were females, and 7 were males.

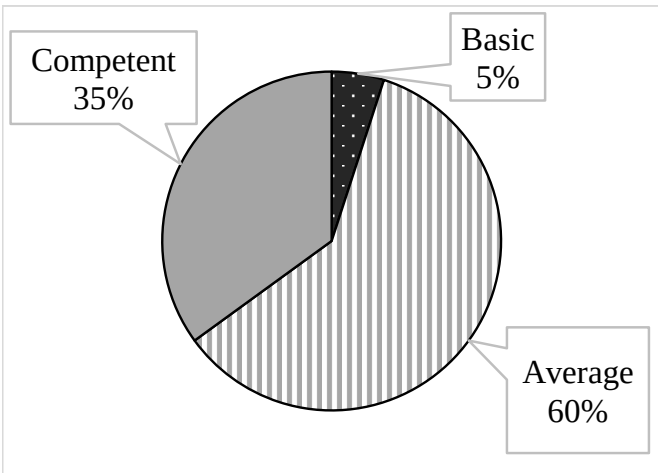

Figure 5: Participants' IT Skills

5.1.2 Apparatus The user study was performed on a Aspire 3 laptop running *Ubuntu 16.04 LTS* in full-screen mode with the *Gnome* environment. The browser used was Google Chrome Version 65.0.3325.181 (Official Build) (64-bit). Trials took place over a one-week period in an open-space laboratory

of the Queen Mother Building, at the University of Dundee in a calm environment at different points of the day and of the week. The system being tested was the version 1.0.0 of *security-comparator*, described in Section 4.

Because the laptop's built-in keyboard was an AZERTY keyboard, an external QWERTY keyboard was used. This is because all the participants were familiar with the latter layout, but not all of them had experience with the first layout. Participants did not seem to notice or be annoyed by the unusual setup, and for that reason it can be expected that the external validity of the results to be affected by this. Figure 6 shows the setup of the experiment.

Figure 6: The setup of the experiment

5.1.3 Procedure Trials were made one participant at a time. If two participants presented themselves at the same time, one of the participants had to stay away for the duration of the first trial with the other participant. This was to prevent the second participant from being influenced or being given an advantage caused by overhearing or seeing the procedure of the experiment. Participants received no compensation and their participation was entirely voluntary. They were made aware of this fact in their invitation as well as at the start of the trial.

Participants were first asked to sit down, read the participant information sheet and asked to sign the consent form. Afterwards, they were asked to write down their age, university course, IT skills and gender on a paper.

They were then told to create an account, log in, and transfer money on *security-comparator*. The money transfer requires the user to prove their identity. *security-comparator* was left open in the web browser on the laptop in front of them. Before they did anything, the author explained to them what was the current security strategy.

Two security strategies were tested: a complex-password-based security strategy (CPWD) and a simple-password- and U2F-based security strategy (SPWD + U2F). In CPWD, the website requires users to have passwords at least 8 characters long, with special characters, uppercase letters, and numbers. The user is then asked to enter their username and password when logging in, and only their password when transferring money. In SPWD + U2F mode, the user is asked to register a U2F Device in addition to a password (without any password complexity requirements) during registration. The user is then asked for their username, password and U2F Device when logging in, and for their password and U2F Device when transferring money. During the user study, *security-comparator* was configured to record the user's performance metrics (i.e. how much time participants spent on each page).

If the website was in CPWD mode, the participant was made aware of the password complexity requirements before he or she began completing the tasks.

If the website was in U2F mode, the use of U2F was explained to the participant in the following manner. U2F was presented as a "physical" password. The registration and the authentication processes were explained by the author. The author visually showed to the participant where and how to plug the U2F Device in (including in what orientation and which slot). The author also insisted on the fact complex passwords were not necessary in SPWD + U2F mode.

Each participant tried one security strategy at a time and filled a NASA-TLX form after each security strategy. Meanwhile, the website was switched to the alternative security strategy by the author. This only involved changing the settings of the website from the same computer and can be performed in a few seconds. The settings could also have been changed from another computer. The participant was then asked to perform the same tasks again on the newly-configured security strategy (after being introduced to the new security strategy in the manner described earlier). Afterwards, the participant was asked to fill another NASA-TLX form.

The ordering of the security strategy was determined in advance by the author. 10 participants tried the SPWD + U2F security strategy first, and the 10 others did it in the reverse order.

Finally, each participant was asked to answer a few questions about their perception of U2F. The questions were:

1. "Did you know about U2F keys before?"
2. "Would you rather prefer being forced to use strong passwords (with at least 8 characters, containing uppercase letters, numbers and special characters), or allowed to use weaker passwords with a U2F key?"
3. "Which method seems the most secure to you?"

5.1.4 Design The experiment had a 1x2 within-subject design. The factor was the security strategy, and its two levels were either CPWD or SPWD + U2F. For each security strategy, the dependent variable was the total time (s) participants needed to complete all the tasks, the registration time (s), and the time (s) it took for the participant to prove their identity when performing the money transfer. This is because the process of proving the user's identity when logging in is the same as the one when transferring money (except *security-comparator* doesn't ask for the username in the second case). Because the money transfer page occurs after the log in page, participants are expected to be familiar with *security-comparator* and with U2F by the time they access the money transfer page. As a result, the performance hindrance caused by learning U2F is diminished. Another dependent variable was the Raw NASA-TLX [13] workload score per security strategy. I refer to the Raw NASA-TLX workload score as the NASA-TLX workload score in the rest of the paper.

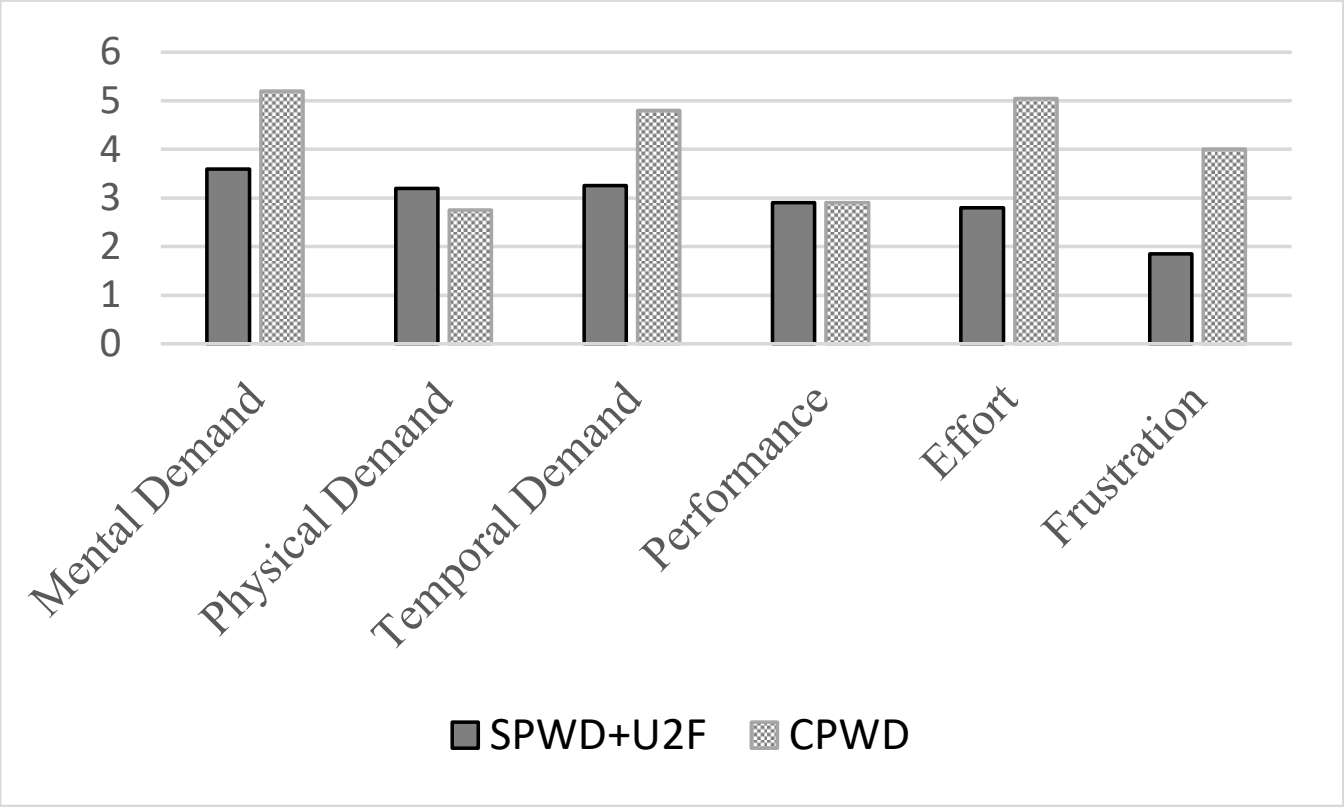

Figure 7: The NASA-TLX Results

5.1.5 Pilot study Two pilot studies with the project supervisor and another anonymous participant were first carried out in order to spot any bug or usability issue before proceeding to the user study. This indeed lead to the detection and correction of several issues, namely:

- Make the system show to the user how to activate their U2F Device.
- Do not ask the user for a name for their U2F Device if the system only asks for one such device.
- Automatically submit the form when the user activates their U2F Device.
- Add a prominent link to the page to the Money Transfer page.
- Do not ask the user to confirm their account registration request. (There used to be a page only meant to display a button that users needed to press to validate the creation of their account.)
- Add an option to prevent members from logging in using their password only. Before, in SPWD + U2F, *security-comparator* allowed participants to choose their login method, and certain users found confusing that they were given the opportunity to choose between password authentication and password + U2F device authentication[20].
- Make the wording more consistent. (Replace ”capital letter” by ”uppercase” letter.)
- Fix a bug with the password validation. (The algorithm did not detect certain special characters.)

[20]This indeed does not make sense in the real world since security is as strong as its weakest link.

5.2 User Performance Metrics Collection

As mentioned in Section 5.1.2, *security-comparator* v1.0.0 was used for running the user study. *security-comparator* v1.0.0 was put online and this is the website that participants used for the user study. At the same time however, progress continued on the development branch of *security-comparator*. These features included the ability to export user performance metrics to CSV files, with the option to filter out redirections. To be able to export the results of the user study into a CSV file (while excluding the redirections), I needed to import the content of the database of the *security-comparator* online installation used for the user study into the database of the latest version of *security-comparator* running locally. However, the latest version had a different database schema. (A new column, ”is_redirection” was added to the Page Metric table.)

Using phpMyAdmin, I added to the *security-comparator* online database a new column to the Page Metric table. The column added was a new nullable, boolean column defaulting to null called ”is_redirection”. This way, I was able to export the page metric data collected from the user study to a SQL file that could be imported into the latest version of *security-comparator*.

Before doing that however, I needed to modify the SQL file. Each exported page metric’s ”is_redirection” value was

set to null, however it needed to be set to either false or true (representing whether it was a redirection or not). In order to do that, I created a PHP script to modify the SQL file to set the value of "is_redirection" depending on the URL associated with the page metric and the preceding and successive URLS. (See Appendix A.)

For example, if the URL of a page metric matches "transfer-money", and the page that comes after is "transfer-money/" followed by a long and random string of characters, this means that the first page was a redirection. This logic is purely application-specific. The author was able to draw these conclusions because he knew this is how the website works. The transformed SQL file was then imported into the latest version of *security-comparator*. From there, the author was able to download CSV representation of the the page metrics for each participant excluding the redirections.

I then combined these CSV files together and put them in one single Excel file containing the rest of the participant data (demographics, NASA-TLX, etc.). The irrelevant pages (i.e. the ones independent of the security strategy used) were removed from the metric (e.g. the homepage, the user confirmation page for the money transfer, the success page, etc). The page metrics were separated in two different sheets, one for each tested security strategy ("upm_pwd" and "upm_u2f", with UPM standing for User Performance Metric). Values were then calculated from these metrics, including the average total time per user, the average registration time per user, and the number of errors per page[21].

5.3 Results

5.3.1 Faster registration On average, excluding the redirections and the irrelevant pages, participants took 68 seconds to complete all the tasks with CPWD and 55 seconds with SPWD + U2F (20% decrease of time). This difference is statistically significant ($F_{1,38} = 7.10, p < 0.05$). Most of the difference lies in the registration process. Participants needed time to come up with a complex password, and sometimes made mistakes (see 5.3.2).

For the authentication process, CPWD seems to be slightly faster (Figure 8). However, there was substantial variation in the observations across participants, and the difference was not statistically significant as revealed in an analysis of variances ($F_{1,38} = 0.6$, ns).

On average, participants took 11.8 seconds to log in using their username and password with CPWD. This is relatively similar to the 8.4 seconds result found by Yakimov [23]. The variation can be explained by the low number of participants in Yakimov's experiment (5 participants), and by the fact CPWD olbiged users to choose long and complex passwords (longer to remember, type, and more error-prone).

5.3.2 Decreased error rate In total, participants made 8 errors with CPWD, and only 1 error in SPWD + U2F (88% less errors). Errors were caused by entering incorrect passwords when registering or when logging in.

[21] The number of errors was determined using an application-specific logic, in a similar fashion than for determining if a page was a redirection or not. E.g. If two many subsequent pages have a URI that match the pattern "login/*" in a certain security strategy, it means there is a certain amount x of errors.

5.3.3 Participant perception of U2F Overall, 80% of the participants preferred a security strategy based on a simple password and a U2F Device rather than one based on complex passwords. NASA-TLX results give an overall workload score of 4.1 for CPWD. For SPWD + U2F, it is 2.9 (29% better). 85% also thought SPWD + U2F was more secure than just CPWD. Also, very importantly, none of the participants knew about U2F Devices. This reveals that the vast majority of users, including computing students, are not aware of U2F. However, a significant amount of effort was put into the development of the FIDO U2F specifications and in ensuring they provide a better usability and user experience.

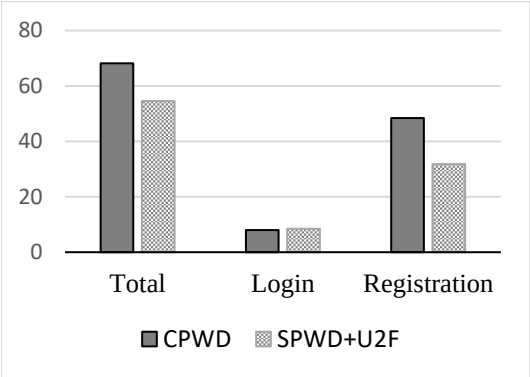

Figure 8: Time (s)

5.4 Initial user study

The user study that was described in this section was the second version. A first version of the user study was made before. 11 participants took part in it. However, the results of the first version are not considered in this paper. This is because the significance of the procedure was too low: It was comparing a security strategy based on complex passwords versus a security strategy based on a U2F Device only (without any passwords). However, U2F is intended to be used as a two-factor authentication mechanism, which means it is designed to be used in addition to passwords to increase online security. So while results were in favour of U2F in the first version of the user study, the results could hardly apply to most websites who would potentially add support for U2F, since these websites would require the user to enter their password as well.

Nevertheless, it wasn't a complete waste of time. The second version of the user study was better designed than the first one with multiple improvements, including:

- In the first version of the user study's setup, the laptop was using an external screen. This confused some of the participants, even when they were experienced in IT. Some tried to plug the U2F Device in the external screen or searched for the computer under the table. In the second version of the user study, the internal screen of the laptop was used, and participants were not confused.
- In the first version of the user study, participants were left alone to figure out how to used U2F Devices. Many were confused and did not know how to do. In the second version of the user study, I explained to the participants how to use U2F Devices before they started using

the website. I did the same for passwords to prevent giving any unfair advantage to U2F.

6 Design and Development Process

6.1 Programming Language

The same programming language was used to develop *security-comparator* and *auth-abstractor*. It would indeed have made little sense to use a different programming language for each of the software product as *auth-abstractor* was designed to be used by *security-comparator* from the ground up. (One motivation behind the development of *security-comparator* is indeed to provide an example and a proof that *auth-abstractor* can be used in a standard web application.)

The programming language I chose was PHP. This is partly because this is a programming language I was very familiar with. The experience I had leveraged my planning abilities for this project as it allowed me to give relatively precise estimates of the time and effort required for the implementation of the features on my backlog. Its latest releases, PHP 7.0 and PHP 7.1, add support for scalar type hints and strict type checking, making the case for PHP even stronger.

In addition, many web applications use PHP [21], especially lightweight web applications [20]. *auth-abstractor* is mainly (although not exclusively) trying to appeal to this kind of web applications. *auth-abstractor* after all acts as a convenience library that abstracts certain features commonly found in web applications (authentication and registration) in order to hide the complexity of these details from developers. In the case of lightweight web applications, it can be expected that developers prefer to spend their limited resources on the features that are specific to their web applications, instead of reinventing the wheel (i.e. implementing an in-house authentication and registration system). Larger web applications usually have the resources needed to handle registration and authentication themselves, and might prefer in-house solutions as it can give them more guarantee and control over the code than an external library.

Of course, choosing a programming language in which I was not familiar could have made for a more formative experience. However, in this project, I was already dealing with technologies that I never used before, including U2F, and Docker in particular. I considered that choosing a programming language that I was not familiar with would not be worth the risk.

Java would also have been a good choice as it is a statically typed programming language that is popular for web development [21]. In regards to security however, the choice between Java and PHP has little impact on the security of the software product developed [21].

6.2 Scrum

I used Scrum to organise my project. I divided the project time into one-month Sprints, and created a backlog for each Sprint. The priority was the release of a new increment each month, even if all the elements on the backlog could not be achieved. However, I neither calculated the velocity of the Sprints, nor attempted to estimate the complexity of each product backlog element using a Planning poker. The velocity of a Sprint is an unreliable measure that is not worth using [22]. In addition, the need for doing planning poker is only justified when Scrum is used within a team, which was not my case. I instead recorded the time it took me to complete each task. This proved to be a reliable metric helping me in monitoring the status of the project and my productivity.

6.3 Continuous Integration

Continuous Integration (CI) is an approach to software development that attempts to streamline the integration process. Fowler [10] describes the methodology in greater details. When using a CI methodology, code is committed often to a central repository. This central repository needs to contain almost everything that is necessary to run the software product, and it must be easy to clone and set up the application. In addition, unit tests are run on each commit to detect bugs as soon as they occur (and to make it easier to fix them). The test environment must be as close as possible to the production environment (e.g. in the case of a website, the server on which the website is hosted).

CI was used for *auth-abstractor* and *security-comparator*. Repositories are easy to set up. In the case of *auth-abstractor*, the only requirement is PHP, Composer and Git. *security-comparator* only requires Docker and Docker Compose. Tests are run on each commit in a test environment using Travis[22]. In addition, the coding style of the code is checked using *php-cs-fixer*[23]. More than just a cosmetic concern, it helps increase the quality of the code, as I configured it to check for the `declare(strict_types=1);` statement at the top of every PHP file. This statement instructs PHP to use a strict typing system, instead of automatically casting values (e.g. by default, PHP would convert `"0"` to `0` if `"0"` is passed to a function that is typed to only accept integers).

Another CI tool that was used in addition to Travis is Scrutinizer[24]. Scrutinizer is a static analysis tool that checks for errors in the code of the software product. This for example can be the assignment of a value to an undefined class property, which is ignored by PHP. It can also detect calls to deprecated functions as well as unused code. It uses the different metrics to calculate the Code Quality of the repository, where 0 means a very bad quality and 10 means that the quality of the code is perfect.

Test Coverage was calculated using Codecov[25]. Test coverage, as its name indicate, is a measure of how well do the unit tests of an application test the application. It is a dynamic analysis tool. PHPUnit, the PHP library that is used for running unit tests, can be configured to generate reports of its execution. It is this report that Dodecov uses to generate the code coverage. Codecov calculates the code coverage of *auth-abstractor* every time a new batch of commits is pushed onto GitHub. Using the metric Codecov provided, I managed to raise the test coverage from 15% to 89%.

Mutation Testing is another metric to obtain the code coverage of an application. It attempts to calculate the MSI, for

[22] `https://travis-ci.org`
[23] `https://github.com/FriendsOfPHP/PHP-CS-Fixer`
[24] `https://scrutinizer-ci.com`
[25] `https://codecov.io`

Mutation Score Index, of an application. The higher the MSI, the better is the application covered by unit tests. I used the PHP library called Infection to calculate the MSI on each commit (using a pre-commit hook). It is also called by Travis. If the MSI relative to the test coverage is lower than 70%, the build fails. This value should be increased in the future.

6.4 Software

6.4.1 Docker At first, *security-comparator* was developed online using the server provided by the University of Dundee. However, the lack of control over the configuration of the server combined with the high latency made me consider an alternative.

As a result, I decided to learn and use Docker. Docker is a virtualisation software that allows the creation and management of virtual environments, called containers, on a host computer. Because the container is separated from the rest of the host machine, there is no risk that the environment of one might affect the environment of the other. This also makes it possible to distribute automatically a product, such as a web application, that works out of the box, without requiring the installation and configuration of new software on the host machine. This is possible because the virtual environment is bundled with the web application within the container. This is the case of *security-comparator*. It is a full-fledged application but it only requires Docker and Docker Compose to run.

Learning Docker and switching my projects to it was challenging for several reasons. First, the performances of Docker on Windows were unsatisfactory, although still much better than on the server of the University of Dundee. In addition, HTTPS needed to be enabled on the container for U2F to work on the web application (see Section 2 for an overview of U2F). Finally, setting up a local database and volumes (disk space shared by both the host machine, on which code is written, and the container, on which code is run) proved challenging.

I installed Ubuntu 16.04 LTS on my laptop in dual-boot with Windows 8.1. Using Ubuntu, the performances of Docker greatly increased. This is because Windows up to 8.1 does not come with a built-in virtualisation engine, which forces the Windows version of Docker, called Docker-Toolbox, to use an external, slower virtual machine (e.g. VirtualBox) with an important negative impact on performances [26].

I managed to enable HTTPS by generating a self-signed certificate from the Docker container. I then configured Apache to use this self-signed certificate and to open the 443 port (the HTTPS port). Because it is a self-signed TLS certificate, web browsers display a warning when first accessing the website. However, it is still possible to access it by adding it as an exception.

The addition of a local database and of volumes was made possible using Docker Compose. This tool makes it easier to use=, manage and connect several Docker containers together, such as a web server container and a database container. Besides, it allows volumes and ports to be defined within a file (”docker-compose.yml”). This is a great improvement over Docker, which requires the user to enter these on the command line every time.

[26] Windows 10 bundles the Hyper-V engine that makes Docker much faster than for its previous versions.

In the end, Docker was worth the time and effort I invested in it. In addition to greatly improving my web development skills, it was very useful in enforcing a specific environment for the application (e.g. a specific version of PHP, Linux, Apache, MariaDB etc.) that could easily be recreated, modified, and shared. It also prevents introducing any bugs caused by a software update or simply by testing one of the projects on a different computer. (e.g. The environment on which Travis is running the tests is identical to the development container). Besides, Docker also makes it possible to use an HTTPS server locally.

6.4.2 Visual Studio Code I used Visual Studio Code. It is a lightweight and free IDE that offers basic features such as code highlighting and integration with Git. However, it is very fast and is very ergonomic. It also defines accelerators for quickly accessing certain files or commands.

6.4.3 Git Git was used as the version control software used. I chose Git because of the experience I had with it, and also because it is widely used by many small and important projects (e.g. the Linux Kernel).

6.4.4 Symfony 4 Symfony 4 was chosen as the web framework to develop *security-comparator*. This is because Symfony was the web framework I was the most experienced with, and also because Symfony 4 was just released in end November 2017, making it one of the most recent PHP web frameworks at the time of this project. It is very-well documented and is bundled with certain very useful features such as a Web Crawler that can be used during testing for simulating interaction with web pages, CSRF protection out-of-the-box (with the component symfony/csrf), routing, form generation and validation, and a templating engine (Twig). It is made up of several, individual components that are used in major projects such as Drupal, Phpbb, and Laravel.

6.4.5 Unit-Testing PHPUnit was chosen to do unit-testing. It also nicely integrates with Travis and Symfony 4. However, I found using a SQLite database for my unit tests challenging. PHPUnit did allow me to specify an SQLite database, but it was not initialised with a schema, making the unit tests fail. As a result, I looked through Symfony 4's source code to find the functions that initialise the database schema, and added what I found to the `setUp()` method of my unit tests. (The `setUp()` method is a method called by PHPUnit before running any test.) As a result, each test was using an SQLite database (which is faster than a traditional database). In addition, the SQLite database is reset between each test, making it impossible that the execution of one test affects the execution of another test that comes after. It also mean running the unit tests would not interfer with the development database.

In order to test pages that handled U2F, I created a file containing several hard-coded U2F registrations, sometimes accompanied with the relevant Register Request, Sign Requests and Sign Response (see Appendix C). This file is parsed and loaded by an instance of `U2fMocker` that can be used in unit tests to simulate a user registering or authenticating.

6.5 Code Design

Because *auth-abstractor* aims at being reusable across different PHP web frameworks, it only communicates with the application that uses it (e.g. *security-comparator*) by accepting and sending back objects implementing interfaces. In particular, for HTTP messages, when possible, PSR-7 standardised interfaces are used[27].

PSRs, for PHP Standard Recommendations, are standards aiming at increasing the interoperability of PHP code made by different developers for different projects. They are developed and maintained by PHP-FIG, a consortium of PHP developers and representatives from popular PHP projects[28]. The standards are widely recognised and applied by the PHP community. For instance, the Symfony coding style enforces the PSR-2 standard, and its container implements the `ContainerInterface` described in the PSR-11 standard.

In addition, I used the Extremely Defensive PHP methodology to write and organise my code [29]. The Extremely Defensive PHP methodology is a school of thought for writing robust PHP code for long-lived, open-source PHP projects. It is advocated by Marco Pivetta, who is also a member of the development team of Doctrine. Objects are made immutable and do not declare any setters. Instead, they are entirely initialised from the constructor. Besides, the constructor only accepts immutable objects, or otherwise clone them. Having only immutable objects make it easier to think about the code and prevent many bugs due to states being updated or values passed by reference being modified. Another principle is that the `protected` scope for class members should never be used. Class inheritance should be avoided by making all classes final by default. Function parameters and return values should always be typed, and never use accept mixed types. A strict type can always be enforced with a Value Object (an object only aimed at validating and representing a value). More generally, I favoured composition over inheritance [11], and attempted to avoid direct references to classes whenever possible to allow for polymorphism (i.e. the use of interfaces).

7 Appraisal and Future Work

The project meets all the goals listed in Section 3, and even supersedes them.

- A library, *auth-abstractor*, was created making it easier for web developers to add full-support for U2F. In addition, the library also supports standard credential-based authentication and registration, and can easily be customised and extended.
- A web application based on this library (*security-comparator*) was developed. It both acts and serves as proof that the library is functional, and it also has its own use. It indeed can be used as a tool for any Human-Computer Interaction researcher willing to compare the usability and user performances of security strategies and authentication mechanisms, and automatically collect metrics.
- A user study was made. Its results prove the superior usability of the security strategy based on a simple password combined with a U2F Device over one purely based on a complex password.

[27] `https://www.php-fig.org/psr/psr-7/`
[28] `https://www.php-fig.org/personnel/`
[29] `https://ocramius.github.io/extremely-defensive-php`

7.1 auth-abstractor

7.1.1 Easiness of use I consider *auth-abstractor* is easy to use for any developer who want to use it in their projects. The code necessary to create a working login page using *auth-abstractor* can take less than 60 lines (Appendix E), which includes the configuration of *auth-abstractor* (which will reused accross other pages such as the registration page).

I created an online survey where I asked developers using PHP at work what they thought of *auth-abstractor*. The participants were invited to take part in the survey with messages sent on developer public chats, facebook groups and mailing lists. 3 of them were also invited directly to take part in the survey (either verbally, or by private message). In all cases, the anonymity of their results was insisted upon and respected. In total, 25 persons answered the survey.

In the survey, a brief textual description of *auth-abstractor* was given along with three code snippets illustrating its use. 95.8% (of the respondents replied ”Yes” to the question: ”Do you get a rough understanding of how you’re supposed to use auth-abstractor?”.

54.2% said they would like to use *auth-abstractor* (some under the condition of a security audit), and 45.8% replied the opposite. However, 857.5% said they believed *auth-abstractor* ”has its use”. 33.3% replied that they thought that there already exists other libraries that do the same thing as *auth-abstractor* but better. 33.3% replied the opposite, and a further 33.3% replied that they were unsure.

Some participants also emitted additional feedback. They were concerned about the lack of test coverage, of an up-to-date documentation, and the absence of a backward compatibility promise.

7.1.2 Test Coverage and Analysis In response, test coverage was added to *auth-abstractor* and increased from 23% to 91%.

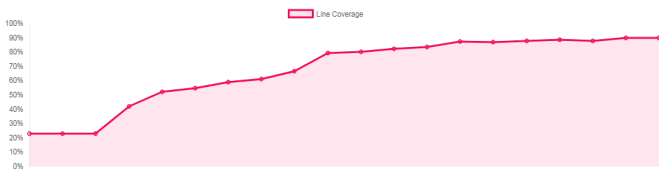

Figure 9: The Evolution of the Test Coverage of the project

The high test coverage (above 90%) combined with a MSI above 70% make provide precise estimate of the code part of the application that is tested and, in other words, proved conform to the requirements.

While the test coverage metric relies on the logs generated by PHPUnit to infer the amount of code and situations that were tested, mutation Testing proceeds completely otherwise. It introduces small changes in the codebase and run the unit tests in order to verify that the unit test fails. The assumption here is that if an application is reasonably covered by unit tests, a

small change in its code would be triggered by some. The library used was Infection. When it is run, Infection calculates the MSI, the mutation score indicator. If the MSI indicates 70%, it means the tests detected mutants in 705 of the case.

In order to ensure the quality of the code at all elve, a pre-commit hook was added to Git. A pre-commit hook is simply a script that gets executed before the commit is already saved. If the script returns 0, the commit is aborted. This is a usefyl feature as it makes it easier to deploy robust code to the central repository, especially if used in conjunction with an external CI tool such as Travis, Codecov or Scrutinizer.

Scrutinizer is another external CI tool that detect use of deprecated code and bad practices. It rated the code of *auth-abstractor* as "very good" with 9.34 out of 10.

The dependencies of both products were submitted to PHP security vulnerabilities monitoring[30], and the service did not detect any vulnerabilities in the dependencies.

Other static analysis was performed using PHPMD, PHPMetrics and QualityAnalyzer. They only found minor potential refactoring advices, but not security vulnerabilities. The results can be accessed online at `https://matthewslouismarie.github.io`.

7.1.3 Documentation Documentation was also added using Phpdocumentor[31], and it can be accessed online [32]. As for the backward compatibility promise, there is none yet, because *auth-abstractor* is still in its infancy and partial redesigns will probably occur in the future. However, once the project will reach a mature state, it will provide a backward compatibility promise.

7.1.4 Application-agnostic *auth-abstractor* is application-agnostic. It communicates with the web application that uses it exclusively with interfaces. When possible, it even uses standardised interfaces, such as the PSR-7 ServerRequestInterface and ServerResponseInterface for representing HTTP request and response. The exclusive reliance on interfaces means it does not depend on any specific storage system, such as a database or the PHP session system. It also means it does not depend on any web framework, such as Symfony or Laravel. In addition, this makes it easier to test individual components in unit tests. In fact, one of *auth-abstractor*'s own unit test mocks a web application using *auth-abstractor* (Appendix F).

7.1.5 Customisable *auth-abstractor* can easily be customised. The views it displays to the user can be overwritten, either partly, entirely, or not at all. The number of maximum attempts per authentication attempt, the password complexity settings, or the number of U2F Devices asked during registration can also be customised. New challenges can easily be added by implemeting the IChallenge interface that *auth-abstractor* provides.

7.1.6 Comparison with other libraries To the best of my knowledge, there exists no other PHP libraries that is application-agnostic provide an abstraction of authentication and registration that includes the generation of an HTTP response (which includes the views), and supports U2F.

Making the library easy to use and application-agnostic was indeed a key priority as it is, along with its support for U2F, its main differentiator from other existing authentication libraries. *Firehed U2F*[33] only provides classes to generate U2F Register and Sign Requests and verify the associated responses. While it is completely application-agnostic, it still requires the web application to handle the entirety of the front-end views as well as the logic of the HTTP controllers.

PHP-Auth[34] provides a more comprehensive abstraction of the authentication process than *Firehed U2F*. However, it is heavily tied to a specific kind of web applications. It indeed strictly requires the use of a specific database schema. This adds up to the overhead of using it in a web application, especially if using an ORM such as Doctrine [35], or if *PHP-Auth* is replacing another authentication system already in place. This also makes it incompatible with applications not using this kind of DBMS, or not using any database at all. It also requires the use of native PHP sessions. Using sessions is sometimes considered a bad practice [8], and not all web applications support, or want to support, native PHP sessions. Native PHP sessions also prevent unit tests from running. Finally, and more importantly, *PHP-Auth* does not support U2F, and leaves the responsibility of handling HTTP requests and generating HTTP responses (including the views) to the web application.

FOSUserBundle[36] is very simple to use and is bundled with pre-made templates for displaying views. However, it does not have support for U2F and more importantly, it only works with Symfony.

7.1.7 Future work *auth-abstractor* does provide all the features that were specified by the initial goals of the project and even more than that (since it also provides password authentication, as well as credentials and U2F registration, and email two-factor authentication). Many improvements could however been carried upon the current version of *auth-abstractor*.

Adding support for the newly upcoming WebAuthn API specification from the WCAG as well as performing more security analysis on the code are among the tasks that could be done to the library. The use of standardised interfaces would also improve make interoperability between existing and future PHP libraries that support U2F. However, the improvement of the PHP implementation of U2F used by *auth-abstractor* was deemed out-of-scope for this project.

The code quality of *auth-abtractor* could be slightly improved by defining more strict coding style standards in PHP-CS-Fixer. The part of the code of *auth-abstractor* that interacts with the application should also implement the PSR-15 standards, which defines a standardised interface for HTTP middlewares.

[30] `https://security.symfony.com`
[31] `https://www.phpdoc.org`
[32] `https://matthewslouismarie.github.io`
[33] `https://github.com/Firehed/u2f-php`
[34] https://github.com/PHPAuth/PHPAuth
[35] Doctrine, a popular PHP Object-Relational Mapping tool used in Symfony, automatically generates a database schema from PHP classes. It is designed to remove from the web application the burden of manually *mapping* the OOP representation of the business domain to a database schema by doing it automatically. This means that the web application never has to directly interact with the database (e.g. using a query language), and as such never defines any table or column name directly.
[36] `https://github.com/FriendsOfSymfony/FOSUserBundle`

7.2 security-comparator

7.2.1 Conformance to original goals The web application developed in this project was originally merely supposed to serve as an example of a web-application that uses *auth-abstractor*. This was both a way to prove that the library does actually work, and also to have a website on which to run a user study to compare the usability of different security strategies. However, it turned out the web application could actually be given a more general purpose and be of more interest. *security-comparator* eventually became a web application which goal is to allow Human-Computer Interaction researchers to run user studies to compare the usability and efficiency of different security strategies. In order to meet this goal several features were added, such as the possibility to import or export the security strategy specification of *security-comparator* which makes studies easily peer-reviewable, and the automatic collection of user performance metrics and the ability to download them as a CSV file. *security-comparator* supports credential-based and U2F-based registration and authentication, U2F Device management, and the Security Strategy can easily be configured from the administration interface. It also allows the calculation of a Security Score for the current Security Strategy.

The calculated Security Score is a very rough estimate. The mathematical model could be greatly improved, for example by using the security properties already defined in existing literature [7].

7.2.2 Support for more 2FA mechanisms *security-comparator* however supports more 2FA mechanisms. This would make it possible for researchers to compare the usability of these additional mechanisms, as well as combinations of theses mechanisms in security strategies (e.g. compare password and U2F Device against password and Google Authenticator). Support for SMS-based 2FA could be added as well. However, sending real SMSs would be expensive. SMS-based 2FA could be mocked by creating a mobile application that behaves as if the participant's phone was receiving SMSs.

7.2.3 Better metrics visualisation *security-comparator* currently has an administration panel named "Visualise user performance metrics" from which the investigator can see in a table the user performance metrics and download them as a CSV file. One useful feature that could be added would be the ability to filter the metrics displayed, e.g. restricting them to certain users, to a certain time interval, etc.). From there, the investigator would be able to download a CSV file of all the metrics matching their search criteria. Adding graphs and diagrams of the generated data could also be a nice addition. However, it is far from being a priority as *security-comparator* is indented to be used as a research tool. This means its users, who are supposed to be researchers, probably intend to do their own analysis and visualisation of the data they collect from users using specialised software such as Excel anyway.

7.2.4 Usability and security heuristics HCI predictive models exist to attempt to predict the time required for a user to complete a certain set of tasks on a computer based on the number of keystrokes, cognitive activities, and mouse pointing operations that they perform.

The score could be shown in the administration panel of *security-comparator* and would help website developers and security strategy designers in designing secure and usable security strategies. This way, *security-comparator* would help in the design and evaluation process (What Security Strategy to adopt?), while *auth-abstractor* would help in the implementation process (How can I implement my Security Strategy?).

7.3 Development approach

7.3.1 Test-Driven Development Test-Driven Development (TDD) was used in the development process of this project. This proved to be very useful for two reasons. First of all, it made it easier to introduce new features and make changes to the product (either *auth-abstractor* or *security-comparator*) while retaining a certain confidence that it does not introduce new bugs. It also provided the other benefits that TDD brings, such as making it easier for the developer to focus on the features of the system. Unit tests (along with the coding style check) were run after each upload of the code to GitHub, and Travis sent me emails whenever a test did not passe.

Behaviour-Driven Development could also have been used. It might have helped defining the requirements for each product. More importantly, it would have helped potential users understand how they are supposed to use each product. However, BDD is a tool primarily aimed at developers who are working with computer novice clients to help them define and enforce the requirements, and this was not my case. As a result, I did not make use of it.

7.3.2 Incremental changes I sometimes made the mistake of making big changes to the code at once (e.g. for refactoring), leaving the fixing of broken unit tests for later. This generally turned out to be a bad idea. I lost time as I was not able to fix the broken unit tests and restore the broken features. I once spent two days trying to fix the application before reverting it to a previous commit, before the refactoring process. After this incident I decided to only introduce small, iterative changes to the repository, and to run unit tests afterwards to determine if commits should be allowed or aborted.

If I am introducing a new feature aimed at replacing an already-existing one, I first add the feature to the product without breaking anything, run unit tests and commit, and then proceed to the removal of the obsolete features (and run unit tests and commit).

7.3.3 Updating I made the mistake of updating my OS (Ubuntu). When the update completed, I could not connect to the university Wi-Fi network anymore. This was caused by a new feature of Ubuntu known as "Mac address randomisation". Unfortunately, this feature prevent the Wi-Fi connection from working on certain networks. Fortunately, I managed to fix the problem. Likewise, I accidentally updated my Composer dependencies without realising it. This broke the rest of the code, but it was easy to reset. It is very important to perform as few updates, OS or otherwise, during a software project, especially as it gets closer to the deadline.

7.4 User study

The user study went relatively well and the results were those expected. (U2F has a better workload index than complex passwords and is perceived to be more secure). However, future user studies could be improved in several ways.

7.4.1 Testing on mobile The benefit of using a simple password and a U2F Device instead of a complex password is expected to be even greater on mobile phones. This is because the process of typing on a mobile phone is generally very frustrating, error-prone, and time-consuming. Activating a U2F Device on a mobile phone however just requires putting them close to each other.

7.4.2 Back-up mechanisms The user study that was run assumed the user would not lose their U2F Device or forget their password. For that reason, the process of registering a back-up method, such as an additional U2F Device or an email, was deemed out-of-scope. This would be an interesting feature to test in an additional user study however. Users were indeed generally concerned about the consequences of losing a U2F Device, although the results of the user study still show overwhelming support for U2F (Section 5.3). It is expected that a back-up mechanism based on U2F would be much easier to use and more secure than one based on a recovery email address.

7.4.3 Longitudinal study Finally, a longitudinal study could also be performed on U2F Devices. It could show how user performances increase with time, whether they register a U2F Device when they have the ability to do so, and what they do when they lose their U2F Devices.

8 Conclusion

It is unrealistic to expect users to choose long, complex, different and frequently-changed passwords for their online accounts. Security strategies that try to enforce these policies are often circumvented by users and are not necessarily more secure.

A security strategy based on a combination of a simple password and a U2F Device is both more secure and more usable. It is a perfect fit for low and medium-security operations, such as logging in, transferring small sums of money, and managing accounts on non-security-critical online services.

Despite dating from end 2014, U2F still remains widely unknown both from developers and from users. The solution is to create more incentive and make it easier to add support for U2F. This project consisted in the development of a library for easily adding support for U2F (as well as passwords and email) authentication and registration to new or existing websites. Another contribution of this project is *security-comparator*, a web application using *auth-abstractor* to make it possible to run and peer review user studies to evaluate and compare the usability of different security strategies. Using *security-comparator*, a user study was tun and it whosed the superior usability of a security strategy based on a simple password and a U2F Device over one solely based on a complex password.

Many more improvements and additions could be done in the future, including adding support for more 2FA mechanisms in *auth-abstractor* and *security-comparator*, and improving the collection and analysis of user performance metrics in *security-comparator*.

9 Acknowledgement

I would like to thank my supervisor Dr. Saša Radomirović for the precious guidance he offered to me during the time of this project. I would also like Anthony Grimes for checking for grammar mistakes in my report. Finally, I would also like to thank all the participants who accepted to take part in the user study or in the survey, P56 in particular.

10 Appendices

Appendices

A transformdata.php

transformdata.php is the PHP script that was used to set redirection information to the SQL file exported from the online database. It can be found at *security-comparator/symfony*.

B Sprint Backlogs

The Sprint Backlogs used for the project. They do not exactly follow the format of a standard Sprint Backlog. The reason is explained in Section 6.2. They can be found in `sprints`.

C u2f_registrations.json

It can be found in the `auth-abstractor` folder.

D Gantt Chart

The Gantt chart is the file `ganttchartt.pdf`.

E One-Page Example of *auth-abstractor*

The Symfony HTTP controller `TmpController` allows the user to log in using *auth-abstractor*. It can be found in `security-comparator/symfony/src/Controller/TmpController.php`.

F Mocked web application in *auth-abstractor*

The AuthenticationKernelTest test suite runs its test by mocking a web application. It can be found in `auth-abstractor/tests`.

G Security Score Calculation

Each challenge definition is an implementation of IChallengeDefinition. Their individual security score is combined by SecurityScoreCalculator. The operations are unit-tested. Challenges are in `security-comparator/symfony/src/Model/{IChallengeDefinition, PwdChallengeDefinition,U2fChallengeDefinition}.php`. `IChallengeDefinition` contains the documentation out of the three. The Security Score Calculator is in `security-comparator/symfony/src/Service/SecurityScoreCalculator.php` and is documented. The importance of each property is determined by the class constants in SecurityScoreCalculator. The class is tested in filesecurity-comparator/symfony/tests/SecurityScoreTest.php.

References

[1] Abawajy, J. User preference of cyber security awareness delivery methods. *Behaviour & Information Technology 33*, 3 (2014), 237–248.

[2] Baghdasaryan, D., and Hill, B. Fido security reference. *FIDO Alliance Proposed Standard* (2017).

[3] Baghdasaryan, D., Hill, B., and Hodges, J. Fido technical glossary. *FIDO Alliance Proposed Standard* (2017).

[4] Balfanz, D. Fido u2f implementation considerations. *FIDO Alliance Proposed Standard* (2017).

[5] Balfanz, D., Birgisson, A., and Lang, J. Fido u2f javascript api. *FIDO Alliance Proposed Standard* (2017).

[6] Balfanz, D., Ehrensvard, J., and Lang, J. Fido u2f raw message formats. *FIDO Alliance Proposed Standard* (2017).

[7] Bonneau, J., Herley, C., Van Oorschot, P. C., and Stajano, F. The quest to replace passwords: A framework for comparative evaluation of web authentication schemes. In *Security and Privacy (SP), 2012 IEEE Symposium on* (2012), IEEE, pp. 553–567.

[8] Denis, F. Thoughts on php sessions. `https://00f.net/2011/01/19/thoughts-on-php-sessions/`, 2011.

[9] Dmitrienko, A., Liebchen, C., Rossow, C., and Sadeghi, A.-R. On the (in) security of mobile two-factor authentication. In *International Conference on Financial Cryptography and Data Security* (2014), Springer, pp. 365–383.

[10] Fowler, M. Continuous integration. *Thought-Works) http://www. thoughtworks. com/Continuous Integration. pdf 122* (2006), 14.

[11] Freeman, E., Robson, E., Bates, B., and Sierra, K. *Head First Design Patterns: A Brain-Friendly Guide*. " O'Reilly Media, Inc.", 2004.

[12] Gollmann, D. *Computer security*, 3rd ed. ed. Wiley ; John Wiley [distributor], Hoboken, N.J. : Chichester, 2011.

[13] Hart, S. G. Nasa-task load index (nasa-tlx); 20 years later. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting 50*, 9 (2006), 904–908.

[14] Hill, B., Balfanz, D., and Baghdasaryan, D. Fido appid and facet specification. *FIDO Alliance Proposed Standard* (2017).

[15] Lang, J., Czeskis, A., Balfanz, D., Schilder, M., and Srinivas, S. Security keys: Practical cryptographic second factors for the modern web. In *International Conference on Financial Cryptography and Data Security* (2016), Springer, pp. 422–440.

[16] Laubheimer, P. Beyond the nps: Measuring perceived usability with the sus, nasa-tlx, and the single ease question after tasks and usability tests. `https://www.nngroup.com/articles/measuring-perceived-usability/`, 2018.

[17] Pearman, S., Thomas, J., Naeini, P. E., Habib, H., Bauer, L., Christin, N., Cranor, L. F., Egelman, S., and Forget, A. Let's go in for a closer look: Observing passwords in their natural habitat. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (2017), ACM, pp. 295–310.

[18] Rabkin, A. Personal knowledge questions for fallback authentication: Security questions in the era of facebook. In *Proceedings of the 4th symposium on Usable privacy and security* (2008), ACM, pp. 13–23.

[19] Srinivas, S., Balfanz, D., Tiffany, E., and Czeskis, A. Universal 2nd factor (u2f) overview. *FIDO Alliance Proposed Standard* (2017).

[20] Trent, S., Tatsubori, M., Suzumura, T., Tozawa, A., and Onodera, T. Performance comparison of php and jsp as server-side scripting languages. In *Proceedings of the 9th ACM/IFIP/USENIX International Conference on Middleware* (New York, NY, USA, 2008), Middleware '08, Springer-Verlag New York, Inc., pp. 164–182.

[21] Walden, J., Doyle, M., Lenhof, R., and Murray, J. Idea: Java vs. php: Security implications of language choice for web applications. In *Engineering Secure Software and Systems* (Berlin, Heidelberg, 2010), F. Massacci, D. Wallach, and N. Zannone, Eds., Springer Berlin Heidelberg, pp. 61–69.

[22] Wolpers, S. Agile metrics — the good, the bad, and the ugly. `https://medium.com/swlh/agile-metrics-the-good-the-bad-and-the-ugly-65639d2` 2016. Accessed: 2018-04-30.

[23] Yakimov, V. Examining and comparing the authentication methods for users in computer networks and systems, 2011.